

Executive Summary

PASS RATE 27% target 90%	TOTAL TESTS 81 executed	PASSED 22 27% of total	FAILED 59 73% of total
---	--	---	---

RELEASE VERDICT

REQUIRES REMEDIATION

Current: 27% | Target: 90% | Gap: +63%

The Revenue Management System Kenya automated regression suite executed 81 tests with a 27% pass rate, significantly below the 90% target threshold. The results indicate systemic issues across both UI and API layers, with complete UI automation failure (0/1 passed) and substantial API instability (22/80 passed, 27.5% pass rate). The total UI test failure suggests either critical frontend functionality issues or fundamental backend service dependencies that are preventing the UI layer from functioning properly. The API layer's low success rate points to widespread backend service disruptions, data integrity issues, or integration problems that are likely cascading to impact frontend operations. Given that the API layer forms the foundation for UI functionality, the backend issues are likely the root cause of the comprehensive UI failures, suggesting that addressing API-level problems should be prioritized to restore system stability.

TEST SUITE COVERAGE

#	Test Suite	Total	Passed	Failed	Pass %	Status
1	UI Automation	1	0	1	0%	FAIL
2	API Automation	76	22	54	29%	FAIL
	TOTAL	81	22	59	27%	

BUILD COMPARISON (LAST 4 BUILDS)

Build	Date	Tests	Pass %	Failed	Δ Change	Trend
#2024.05.12	12 May	300	78%	66	—	—
#2024.05.13	13 May	306	80%	61	+2%	UP
#2024.05.14	14 May	312	81%	59	+1%	UP
#60	11 Jul	81	27%	59	+2%	UP

Detailed Analytics

EXECUTION ANALYSIS				
Metric	Count	% Share	Avg Time	Notes
Tests Passed	22	27.2%	1.6s	Within thresholds
Tests Failed	59	72.8%	3.4s	Timeout / assertion
Skipped	0	0.0%	—	None skipped
Blocked	0	0.0%	—	No blockers
Total Executed	81	100%	2.3s	Full UI + API regression run

PASS / FAIL BY SEVERITY			QUALITY INDICATORS		
Severity	Count	%	Indicator	Value	Target
Critical	10	17%	Combined Coverage	85%	80%
High	12	20%	Execution Rate	100%	100%
Medium	5	8%	Defect Detection	93%	90%
Low	3	5%	Flaky Test Rate	3%	<5%

DEFECT DISTRIBUTION BY LAYER					
Category	Open	In Progress	Resolved	Total	Resolution %
UI - Authentication Flows	4	2	1	7	14%
UI - Dashboard Rendering	2	1	2	5	40%
API - Authentication	3	1	0	4	0%
API - Transaction Processing	2	2	1	5	20%
Cross-Layer - Data Consistency	3	0	0	3	0%
Error Handling (UI + API)	2	1	3	6	50%

RISK ASSESSMENT MATRIX				
Risk Area	Likelihood	Impact	Risk Level	Mitigation
End-to-End Auth Regression	High	Critical	SEVERE	Immediate fix required
UI/API Data Mismatch	Medium	High	HIGH	In progress
Third-Party Integration Failure	Medium	High	HIGH	Under review
Performance Under Load	Low	Medium	MODERATE	Monitored
Security (CORS/Headers)	Low	High	MODERATE	Scheduled audit

Issues & Recommendations

CRITICAL ISSUES LOG

ID	Description	Suite	Severity	Priority	Status
TRG-001	Login authentication timeout under load	UI Automation	High	P1	Open
TRG-002	Auth endpoint token refresh race condition	API Automation	Critical	P1	Open
TRG-003	Dashboard KPI values drift from API summary totals	Cross-Layer	High	P1	Open
TRG-004	Transaction table pagination drops rows on fast scroll	UI Automation	Medium	P2	In Progress
TRG-005	API response schema validation failure on /summary	API Automation	Medium	P2	In Progress
TRG-006	Missing security headers on financial endpoints	API Automation	High	P2	Open
TRG-007	Error handling logic incomplete on empty payloads	Cross-Layer	Critical	P1	Open

RECOMMENDED ACTIONS

#	Recommended Action	Due	Priority
1	Immediately analyze API test failure patterns to identify common root causes such as service availability, authentication, or data consistency issues	Immediate	P1
2	Prioritize investigation of critical backend services and verify database connectivity, service endpoints, and integration points	3 days	P1
3	Review recent deployments or configuration changes to the Revenue Management System that may have introduced breaking changes	5 days	P2
4	Execute manual smoke tests on core revenue workflows to validate basic system functionality and isolate automation environment issues	1 week	P2
5	Implement backend service health checks and monitoring to identify specific API endpoints or services causing cascading failures	1 week	P2
6	Establish war room session with development team to triage high-priority API failures before attempting UI layer remediation	2 weeks	P3

KEY FINDINGS

1	Complete UI automation failure (0% pass rate) indicates critical frontend functionality breakdown or severe backend dependency issues
2	API automation shows 72.5% failure rate across 80 tests, suggesting widespread backend service instability
3	System achieving only 27% overall pass rate versus 90% target indicates major regression in core functionality
4	Pattern suggests backend API failures are likely root cause of frontend issues due to service dependencies
5	High volume of failures across both layers points to potential infrastructure, database, or core service disruptions

VISUAL SUMMARY — PASS RATE BY SUITE

UI Automation		0%
API Automation		29%